UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

GENEROVANIE 3D MODELOV POMOCOU VEĽKÉHO JAZYKOVÉHO MODELU

Diplomová práca

2026 Bc. Filip Béreš

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

GENEROVANIE 3D MODELOV POMOCOU VEĽKÉHO JAZYKOVÉHO MODELU

Diplomová práca

Študijný program:	Aplikovaná informatika
Odbor štúdia:	Informatika
Pracovisko:	Katedra aplikovanej informatiky
Vedúci práce:	Mgr. Andrej Mihálik, PhD.

Bratislava, 2026 Bc. Filip Béreš

41332642

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Filip Béreš
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Generovanie 3D modelov pomocou veľkého jazykového modelu
3D Model Generation with a Large Language Model

Anotácia: Diplomová práca sa zameriava na vytvorenie systému, ktorý dokáže z textového zadania v prirodzenom jazyku automaticky vygenerovať trojrozmerný model v programe Blender. Kľúčovým prvkom riešenia je využitie veľkého jazykového modelu (LLM), ktorý bude lokálne nainštalovaný a prispôsobený na oblasť 3D modelovania.

Systém bude fungovať tak, že jazykový model rozloží zložitejší objekt na jednotlivé časti, opíše ich základné parametre a tieto údaje sa následne spracujú v Pythone a prenesú do Blenderu. Celý proces – od textového zadania cez odpoveď modelu až po finálny 3D objekt – bude testovaný na viacerých príkladoch.

Výsledkom práce bude ukážka, ako možno pomocou veľkého jazykového modelu a jednoduchej softvérovej pipeline premeniť prirodzený jazyk na vizuálny 3D objekt, čo otvára nové možnosti pre kreatívne využitie umelej inteligencie v dizajne a digitálnom umení.

Vedúci: Mgr. Andrej Mihálik, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.

Dátum zadania: 07.12.2025

Dátum schválenia: 08.12.2025

prof. RNDr. Roman Ďurikovič, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

Čestne vyhlasujem, že celú diplomovú prácu na tému Generovanie 3D modelov pomocou veľkého jazykového modelu, vrátane všetkých jej príloh a obrázkov, som vypracoval samostatne, a to s použitím literatúry uvedenej v priloženom zozname a nástrojov umelej inteligencie. Vyhlasujem, že nástroje umelej inteligencie som použil v súlade s príslušnými právnymi predpismi, akademickými právami a slobodami, etickými a morálnymi zásadami za súčasného dodržania akademickej integrity a že ich použitie je v práci vhodným spôsobom označené.

Bratislava, 2026

. . . . . .

Bc. Filip Béreš

Poďakovanie

Abstrakt

Slovenský abstrakt v rozsahu 100-500 slov, jeden odstavec. Abstrakt stručne sumarizuje výsledky práce. Mal by byť pochopiteľný pre bežného informatika. Nemal by teda využívať skratky, termíny alebo označenie zavedené v práci, okrem tých, ktoré sú všeobecne známe.

Kľúčové slová: jedno, druhé, tretie (prípadne štvrté, piate)

Abstract

Abstract in the English language (translation of the abstract in the Slovak language).

Keywords:

Obsah

Úvod **1**

1 Teoretické základy **3**

1.1 Veľké jazykové modely 3

1.2 Techniky promptovania 4

1.2.1 Few-shot learning 4

1.2.2 Chain-of-Thought a rozklad úloh 5

1.2.3 Retrieval-Augmented Generation 5

1.3 Prehľad nástrojov 5

1.3.1 Blender a Python API 5

1.3.2 Ollama 6

1.4 Súvisiace práce 6

1.4.1 3D-GPT 7

1.4.2 LL3M 7

1.4.3 Porovnanie a pozícia tejto práce 8

2 Návrh systému **9**

2.1 Požiadavky a obmedzenia 9

2.2 Schéma navrhnutého systému 9

2.3 Formát výmeny medzi agentmi 9

2.4 Voľba jazykového modelu 9

3 Implementácia **11**

3.1 Conceptualizer 11

3.2 Planner 11

3.3 Modeler 11

3.4 Checker 11

3.5 Spracovanie odpovedí jazykového modelu 11

3.6 Generovanie Blender skriptu 11

4 Experimenty a výsledky **13**
4.1 Testovacia sada 13
4.2 Generované objekty 13
4.2.1 Jednoduché objekty 13
4.2.2 Zložené objekty 13
4.2.3 Hraničné prípady 13
4.3 Vplyv jednotlivých komponentov pipeline 13
4.4 Časová náročnosť generovania 13

Záver **15**

Literatúra **17**

Zoznam obrázkov

Zoznam tabuliek

Úvod

a

Kapitola 1

Teoretické základy

V tejto kapitole zhrnieme základné pojmy, na ktoré budeme nadväzovať v zvyšku práce. Najprv predstavíme veľké jazykové modely (sekcia 1.1), potom techniky, ktorými sa dá ovplyvniť ich správanie pri zadávaní úloh (sekcia 1.2). V sekcii 1.3 popíšeme softvérové nástroje, ktoré sme v práci využili. Kapitolu uzatvára prehľad relevantných existujúcich riešení (sekcia 1.4).

1.1 Veľké jazykové modely

Veľké jazykové modely (LLM, z angl. *Large Language Models*) sú neurónové siete typu transformer, ktoré boli trénované na obrovskom množstve textových dát. Ich úlohou je predikovať ďalšie slovo (presnejšie token) v texte. Hoci ide o veľmi jednoduchú úlohu, modely s miliardami parametrov sa pri jej riešení naučili prekvapivo všeobecné schopnosti. Okrem plynulého písania v prirodzenom jazyku vedia generovať programový kód, riešiť matematické úlohy či krok za krokom uvažovať o zložitých problémoch.

Generovanie prebieha autoregresívne. Model dostane na vstup kontext (zoznam doterajších tokenov) a predikuje pravdepodobnostné rozdelenie nad slovníkom všetkých možných ďalších tokenov. Z tohto rozdelenia sa potom vyberie konkrétny token a celý proces sa zopakuje. Spôsob výberu kontroluje parameter *teplota*. Pri nulovej teplote model vždy vyberie najpravdepodobnejší token, čo robí výstup deterministickým ale často monotónnym. Pri vyšších teplotách (typicky okolo 0,7) má model väčšiu slobodu a výstupy sú rozmanitejšie. V tejto práci pri úlohách, kde nám záleží na štruktúrovanom výstupe (napríklad JSON), používame nízke teploty v rozmedzí 0,2 až 0,5.

Schopnosť LLM produkovať funkčný programový kód vychádza z toho, že tréningové dáta obsahujú veľké množstvo zdrojového kódu z verejných repozitárov, dokumentácie a portálov ako Stack Overflow. Niektoré modely sú dokonca priamo špecializované na túto úlohu – napríklad rodina Qwen2.5-Coder [2] alebo DeepSeek-Coder. Pri generovaní kódu dosahujú výrazne lepšie výsledky než všeobecné modely porovnateľnej

veľkosti. V našej práci využívame túto schopnosť netradičným spôsobom: programový kód nie je výsledný produkt, ale prostredník medzi prirodzeným jazykom a externým nástrojom. LLM napíše Python skript pre Blender a jeho spustením vznikne 3D model.

Hoci najznámejšie modely ako GPT-4 alebo Claude sú proprietárne a dostupné iba cez platené cloudové rozhrania, paralelne existuje aj rodina otvorených modelov. Patria k nim Llama od spoločnosti Meta, Mistral od rovnomennej spoločnosti a Qwen od Alibaby. V posledných rokoch sa kvalita otvorených modelov výrazne priblížila k proprietárnym, najmä pri úlohách generovania kódu. Otvorenosť modelov umožňuje ich *lokálne nasadenie*, čiže model beží priamo na používateľskom počítači a nepotrebuje pripojenie na cloud. To prináša niekoľko výhod, ktoré sú dôležité pre vedeckú prácu. Riešenie je reprodukovateľné, lebo váhy modelu sa nemenia. Nezávisíme od žiadneho poskytovateľa, ktorý by mohol službu zmeniť alebo zrušiť. Dáta neopúšťajú náš počítač. A po prvotnej investícii do hardvéru je každé volanie zadarmo, čo umožňuje rozsiahle experimentovanie. Cenou za tieto výhody je zvyčajne nižšia kvalita výstupov v porovnaní s najväčšími proprietárnymi modelmi a vyššie nároky na lokálny hardvér – typicky niekoľko gigabajtov videopamäte.

1.2 Techniky promptovania

Kvalita výstupov LLM nezávisí len od samotného modelu, ale aj od toho, ako mu úlohu naformulujeme. Súhrnný pojem *prompt engineering* označuje techniky, ktorými sa dá výrazne zlepšiť spoľahlivosť a presnosť odpovedí bez akéhokoľvek zásahu do váh modelu. V tejto sekcii popíšeme tri techniky, ktoré priamo využívame v našej pipeline.

1.2.1 Few-shot learning

V strojovom učení pojem *few-shot* označuje učenie z malého počtu označených príkladov. V kontexte veľkých jazykových modelov má však tento výraz špecifický význam, ktorý popularizoval článok [1]: do promptu zahrnieme niekoľko (typicky 1 až 5) hotových príkladov požadovaného vstupu a výstupu, a model ich pri generovaní napodobní.

V praxi je tento prístup mimoriadne účinný pri úlohách s pevne danou štruktúrou výstupu. Ak chceme od modelu napríklad JSON s konkrétnou schémou, samotný verbálny popis schémy často nestačí. Aj keď v prompte presne vymenujeme všetky polia a ich typy, menší model ich napriek tomu môže ignorovať alebo zameniť. Pridanie jediného konkrétneho príkladu však tento problém zvyčajne odstráni. Túto techniku používame v našej pipeline pri generovaní JSON popisu objektu (viď kapitolu 3).

1.2.2 Chain-of-Thought a rozklad úloh

Technika *chain-of-thought* (CoT) [6] spočíva v tom, že model nútime riešiť úlohu krok za krokom, namiesto toho, aby skočil priamo k odpovedi. Autori článku experimentálne ukázali, že LLM dosahujú výrazne lepšie výsledky pri úlohách, ktoré vyžadujú viacstupňové uvažovanie – napríklad pri matematických problémoch alebo logických hádankách – ak ich pred odpoveďou vyzveme, aby si rozpísali postup.

Tento princíp sa však dá uplatniť aj na vyššej úrovni – na úrovni celej architektúry systému. Namiesto toho, aby sme sa LLM pýtali na všetko v jednom prompte, môžeme úlohu rozdeliť medzi viacero volaní, pričom každé volanie rieši jednu jasne vymedzenú podúlohu. Tento prístup vedie k tzv. *multi-agentnej architektúre*, ktorú dnes používa väčšina pokročilých systémov v oblasti, vrátane prác popísaných v sekcii 1.4.

1.2.3 Retrieval-Augmented Generation

Veľké jazykové modely majú dve známe slabiny: ich znalosti sa končia dátumom uzávierky tréningových dát, a majú tendenciu *halucinovať*, čiže vymýšľať si fakty, ktoré znejú vierohodne, ale nezodpovedajú realite. Technika *retrieval-augmented generation* (RAG) [3] sa snaží obe tieto slabiny adresovať.

Princíp RAG je nasledovný. Popri otázke od používateľa dostáva model aj relevantné fragmenty z externej znalostnej bázy, typicky korpusu dokumentácie alebo databázy. Tieto fragmenty sa vyberajú dynamicky podľa sémantickej podobnosti s otázkou. Podobnosť sa meria pomocou tzv. *vektorových embeddingov*: každý text je reprezentovaný vysokorozmerným vektorom a podobnosť medzi dvoma textami je daná vzdialenosťou medzi ich vektormi. Vďaka tomu má model v reálnom čase prístup k aktuálnym a overeným informáciám a nemusí sa spoliehať iba na svoju statickú pamäť.

V kontexte našej práce je RAG zaujímavý preto, že jeden z existujúcich systémov pre generovanie 3D objektov (LL3M, podsekcia 1.4.2) ho používa nad dokumentáciou Blender Python API. Tým dokáže výrazne znížiť frekvenciu halucinácií, pri ktorých model vymýšľa neexistujúce funkcie. V našej pipeline RAG zatiaľ nevyužívame, hoci ho zvažujeme ako možné rozšírenie.

1.3 Prehľad nástrojov

1.3.1 Blender a Python API

Blender je voľne dostupný profesionálny nástroj pre 3D tvorbu. V posledných rokoch sa stal jedným z hlavných nástrojov v produkcii animácie, hier a vizuálnych efektov. Obsahuje kompletnú sadu funkcií pre modelovanie, textúrovanie, animáciu, simuláciu fyziky aj renderovanie.

Pre účely tejto práce je dôležité, že Blender ponúka aj kompletné programovacie rozhranie v jazyku Python, dostupné cez modul `bpy`. Cez toto API sa dá programovo vykonať takmer všetko, čo je v grafickom rozhraní dostupné myšou. Napríklad volanie:

```
bpy.ops.mesh.primitive_cube_add(location=(0, 0, 1))
```

pridá do scény novú kocku v zadanej polohe – presne ako keby používateľ klikol v hlavnom menu na *Add* → *Mesh* → *Cube*. Python skripty sa dajú spustiť priamo v Blenderi (cez pracovný priestor *Scripting*) alebo z príkazového riadku v tzv. bezhlavom režime, čo umožňuje úplnú automatizáciu.

1.3.2 Ollama

Ollama je nástroj, ktorý zjednodušuje lokálne nasadenie otvorených jazykových modelov. Beží ako lokálny server (štandardne na porte 11434) a poskytuje HTTP rozhranie, cez ktoré sa klientske aplikácie pýtajú modelu. Knižnica modelov je centrálne udržiavaná a jednotlivé modely sa sťahujú jediným príkazom, napríklad:

```
ollama pull qwen2.5-coder:7b
```

Z technického hľadiska Ollama používa formát GGUF, ktorý uchováva modely v kvantizovanej podobe – pôvodné parametre uložené v 32-bitovej presnosti sa prevedú na nižšiu presnosť (typicky 4-bitovú). Kvantizácia výrazne znižuje veľkosť modelu aj pamäťové nároky pri behu, pričom strata kvality výstupu je v praxi zvyčajne malá. Pre konkrétny príklad, model Qwen2.5-Coder so 7 miliardami parametrov v kvantizácii `Q4_K_M` zaberá približne 5 GB videopamäte, čo sa zmestí do bežnej hernej grafickej karty.

Z hľadiska vývoja je Ollama užitočná najmä tým, že vie cez parameter `format="json"` vynútiť, aby model produkoval syntakticky platný JSON. Pre úlohy ako generovanie štruktúrovaných popisov 3D objektov je táto vlastnosť mimoriadne praktická.

1.4 Súvisiace práce

Automatické generovanie 3D modelov z textového popisu je oblasť, ktorá v posledných rokoch zaznamenala výrazný rozvoj. Existujúce riešenia sa rámcovo dajú rozdeliť do troch kategórií.

Prvú tvoria *generatívne neurónové reprezentácie*, ktoré optimalizujú parametre neurónového radiačného poľa tak, aby jeho 2D renderingy zodpovedali vizuálnym predstavám difúzneho modelu. Výsledky bývajú vizuálne pôsobivé, ale výstupom je hustá implicitná reprezentácia, ktorá sa ťažko upravuje v bežných grafických nástrojoch, a generovanie jediného objektu trvá desiatky minút.

Druhú skupinu tvoria prístupy, ktoré sa opierajú o existujúce procedurálne generátory a LLM používajú iba na odvodenie hodnôt parametrov. Typickým príkladom je práca 3D-GPT popísaná v podsekcii 1.4.1.

Tretia, najnovšia línia výskumu formuluje úlohu generovania 3D objektu ako úlohu *písania programového kódu*. Sem patrí napríklad práca LL3M (podsekcia 1.4.2). Práve do tejto tretej kategórie patrí aj naša diplomová práca.

1.4.1 3D-GPT

Práca 3D-GPT [5] bola jednou z prvých, ktoré priniesli koncept využitia LLM na riadenie procedurálneho generovania 3D scén v Blenderi. Autori vychádzajú z pozorovania, že procedurálne generátory dokážu vytvárať vizuálne kvalitné prírodné scény (lesy, hory, jaskyne), ale ich ovládanie vyžaduje znalosť stoviek modifikovateľných parametrov. Riešením je vložiť medzi používateľa a procedurálny systém vrstvu jazykového modelu: používateľ napíše krátku požiadavku v prirodzenom jazyku a LLM zariadi výber vhodných procedurálnych funkcií a doplnenie ich argumentov.

Koncepčným prínosom práce je rozdelenie úlohy medzi tri špecializované agenty, postavené na rovnakom LLM s rôznymi systémovými promptami. *Task Dispatch Agent* prijme používateľský vstup a rozhodne, ktoré procedurálne funkcie sú pre úlohu relevantné. *Conceptualization Agent* obohatí stručný popis o vizuálne detaily, ktoré používateľ neuviedol – napríklad typ snehu alebo hustotu vegetácie. *Modeling Agent* z obohateného popisu extrahuje konkrétne číselné hodnoty parametrov a zostaví Python skript, ktorý sa odovzdá Blenderu. Toto rozdelenie zodpovedá princípu chain-of-thought reasoning popísanému v podsekcii 1.2.2.

Hlavné obmedzenia systému, ktoré autori sami priznávajú, sú: viazanosť na slovník použitej procedurálnej knižnice (čo nie je v katalógu funkcií, to nemožno vyprodukovať), zameranie na prírodné scény skôr než na štruktúrované objekty, a závislosť od proprietárnych modelov dostupných výhradne cez komerčné API.

1.4.2 LL3M

Práca LL3M [4] predstavuje jeden z najpokročilejších existujúcich systémov pre generovanie 3D objektov pomocou LLM. Oproti práci 3D-GPT robí niekoľko zásadných posunov.

Najpodstatnejším koncepčným posunom je reformulácia úlohy: LLM nedostáva za úlohu vyberať z katalógu predpripravených funkcií, ale priamo *napísať Python kód* pre Blender. Tým získava prístup k plnej expresivite Blender Python API, vrátane pokročilých konštrukcií ako BMesh, geometry modifiers a shader nodes. Ďalšou výhodou je čitateľnosť výstupu – vygenerovaný kód si umelec môže prečítať a ďalej manuálne upraviť.

Architektúra LL3M je bohatšia než v prípade 3D-GPT a obsahuje päť agentov. *Planner* vytvorí hrubý plán objektu. *Retriever* vyhľadá v dokumentácii Blender API relevantné fragmenty pre plánované operácie. *Coder* na základe plánu a vyhľadanej dokumentácie napíše skutočný kód. *Debugger* reaguje na prípadné chyby pri spustení kódu a opravuje ich. A konečne *Refiner* po úspešnom behu porovná vyrenderovaný obrázok scény s pôvodným zadaním a navrhne ďalšie úpravy.

Najpodstatnejším technickým príspevkom je systém *BlenderRAG* – implementácia princípu RAG (podsekcia 1.2.3) nad dokumentáciou Blender Python API. LLM pri generovaní kódu nepracuje len so svojimi tréningovými dátami, ale má prístup k aktuálnej dokumentácii s popismi tried, funkcií a parametrov. Tým sa významne znižuje frekvencia halucinácií, pri ktorých model vymýšľa neexistujúce funkcie.

Obmedzenia, ktoré autori uvádzajú, zahŕňajú stochastickosť výsledkov (pri rovnakom zadaní môžu rôzne behy priniesť kvalitatívne odlišné výsledky) a závislosť od proprietárneho jazykového modelu.

1.4.3 Porovnanie a pozícia tejto práce

Z porovnania prác 3D-GPT a LL3M vyplýva niekoľko pozorovaní, ktoré určujú smerovanie tejto diplomovej práce. Po prvé, novšia línia výskumu naznačuje, že generovanie kódu ponúka oproti procedurálnym knižniciam výrazne väčšiu flexibilitu, najmä pri objektoch s jasne definovanými časťami. Po druhé, žiadna z uvedených prác sa nevenuje otázke, do akej miery sú tieto výsledky dosiahnuteľné s lokálne nasaditeľným otvoreným jazykovým modelom – obe sa opierajú o proprietárne modely cez cloudové API. Po tretie, plná architektúra LL3M (najmä BlenderRAG a vizuálny Refiner) je príliš robustná na to, aby ju bolo možné v rámci diplomovej práce kompletne replikovať.

Z týchto pozorovaní vychádza zameranie tejto práce. Preberáme z LL3M centrálnu myšlienku reformulácie úlohy ako generovania kódu, aj princíp multi-agentnej dekompozície. Spájame ich s obmedzením, ktoré v ostatných prácach nie je: **celá pipeline má bežať lokálne**, na bežnom hardvéri, s otvoreným jazykovým modelom. Cieľom nie je prekonať LL3M v kvalite výstupov, ale zodpovedať otázku, čo je v tejto oblasti dosiahnuteľné bez závislosti na komerčných API. Konkrétny návrh nášho riešenia predstavujeme v kapitole 2.

Kapitola 2

Návrh systému

2.1 Požiadavky a obmedzenia

2.2 Schéma navrhnutého systému

2.3 Formát výmeny medzi agentmi

2.4 Voľba jazykového modelu

Kapitola 3

Implementácia

3.1 Conceptualizer

3.2 Planner

3.3 Modeler

3.4 Checker

3.5 Spracovanie odpovedí jazykového modelu

3.6 Generovanie Blender skriptu

Kapitola 4

Experimenty a výsledky

4.1 Testovacia sada

4.2 Generované objekty

4.2.1 Jednoduché objekty

4.2.2 Zložené objekty

4.2.3 Hraničné prípady

4.3 Vplyv jednotlivých komponentov pipeline

4.4 Časová náročnosť generovania

Záver

a

Literatúra

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 2020.

[2] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

[3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 2020.

[4] Sining Lu, Guan Chen, Nam Anh Dinh, Itai Lang, Ari Holtzman, and Rana Hanocka. LL3M: Large language 3d modelers, 2025.

[5] Chunyi Sun, Junlin Han, Weijian Deng, Xinlong Wang, Zishan Qin, and Stephen Gould. 3D-GPT: Procedural 3d modeling with large language models, 2023.

[6] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 2022.